CSA07 Computer Networks — Assignment 2: Worked Answers

IP Addressing, Subnetting and Routing (CO2) — Q1 to Q7

Table of Contents

Q1 — Spot the Invalid Addresses....1
Q2 — Subnet a Campus That Keeps Growing....2
Q3 — Which Way Will the Packet Go?....4
Q4 — Watch Distance-Vector Routing Count to Infinity....5
Q5 — Why Does OSPF Treat 1 Gbps and 10 Gbps the Same?....7
Q6 — IPv6, ICMP, Multicast and SDN in a Smart City....8
Q7 — Find the Weak Point, Redesign and Validate....9

GitHub repository: https://github.com/govukrithi-dev/Computer-networks.git

> **How to use this file.** These are worked answers and reference calculations. The assignment also asks for your own Packet Tracer (.pkt) files, router outputs, screenshots and commit history. Build the networks yourself, run the programs in the accompanying code folder, and commit one question at a time.

Q1 — Spot the Invalid Addresses

Device	Mask	Network	Broadcast	Usable hosts	Valid host?
Admission PC 172.16.45.200 /20	255.255.240.0	172.16.32.0	172.16.47.255	172.16.32.1 – 172.16.47.254	**Yes**; gateway 172.16.32.1 is in the same subnet
Lab printer 10.0.15.255 /21	255.255.248.0	10.0.8.0	10.0.15.255	10.0.8.1 – 10.0.15.254	**No — trap 1:** the address is the **broadcast address**
X-ray server 192.168.4.0 /23	255.255.254.0	192.168.4.0	192.168.5.255	192.168.4.1 – 192.168.5.254	**No — trap 2:** the address is the **network address**
Clinic router 203.0.113.10 /29	255.255.255.248	203.0.113.8	203.0.113.15	203.0.113.9 – 203.0.113.14 (6 hosts)	**Yes**

Working: for /20 the block size in the third octet is 16 (45 falls in 32–47); for /21 it is 8 (15 falls in 8–15); for /23 it is 2 (4 falls in 4–5); for /29 it is 8 in the last octet (10 falls in 8–15).

"Run out of addresses": the /29 gives only 6 usable addresses for 18 devices, which is why NAT is needed.

NAT table (port address translation). Two PCs use source port 50000 at the same time; the router gives each a different public port.

Inside local	Inside global (public)	Destination
192.168.1.10 : 50000	203.0.113.10 : 61001	93.184.216.34 : 443
192.168.1.11 : 50000	203.0.113.10 : 61002	93.184.216.34 : 443

Replies come back to ports 61001 and 61002, and the router looks up the table to send each one to the right PC.

Datagram versus virtual circuit. In datagram switching every packet carries the destination address and each router chooses the next hop independently. Routes or queue lengths can differ between packets, so packets of one file can arrive out of order. With virtual circuits a path is set up first; every packet carries a circuit identifier and follows the same path, so order is preserved. The price is set-up delay, state in every switch, and the circuit breaks if a switch on the path fails.

Alternative — a public /27 instead of NAT behind a /29

- A /27 has 32 addresses, 30 usable, enough for the 18 devices (each needs its own public address).
- Cost: 18 public addresses are used instead of 1, so the provider charge is higher and IPv4 addresses are scarce.
- Exposure: every device is reachable from the internet, so a firewall is mandatory.
- NAT problems that disappear: inbound connections and servers without port forwarding, end-to-end protocols (SIP/VoIP, FTP active mode, IPsec), peer-to-peer, and port exhaustion.

Q2 — Subnet a Campus That Keeps Growing

Requirement: Labs 100, Staff 50, Library 25, WAN link 2 hosts in 192.168.10.0/24.

Plan 1 — equal-size subnets. Four equal subnets mean /26 (64 addresses, 62 hosts).

Subnet	Network	Mask	Hosts
1	192.168.10.0/26	255.255.255.192	62
2	192.168.10.64/26	255.255.255.192	62
3	192.168.10.128/26	255.255.255.192	62
4	192.168.10.192/26	255.255.255.192	62

This **fails**: Labs needs 100 hosts but a /26 holds only 62, and the Library wastes 37 addresses and the WAN link wastes 60. Using /25 for all gives only two subnets. Equal sizing cannot serve these requirements.

Plan 2 — VLSM (largest first)

Subnet	Hosts needed	Block	Network / prefix	Mask	First host	Last host	Broadcast
Labs	100	128	192.168.10.0/25	255.255.255.128	.1	.126	.127
Staff	50	64	192.168.10.128/26	255.255.255.192	.129	.190	.191
Library	25	32	192.168.10.192/27	255.255.255.224	.193	.222	.223
WAN link	2	4	192.168.10.224/30	255.255.255.252	.225	.226	.227

Next free address: **192.168.10.228**. Free space: 28 addresses (.228 to .255).

Trap — the 60-host hostel. It needs a /26 (64 addresses). Only 28 addresses remain, and the largest aligned free block is a /28 (16 addresses), so it **does not fit**. Total demand is now 128 + 64 + 32 + 4 + 64 = 292, more than the 256 addresses of the /24. **Smallest change:** give the hostel the next block, **192.168.11.0/26**, and treat the campus as 192.168.10.0/23 (one summary route). No existing subnet is renumbered.

Route summarisation

- 172.16.8.0/24 to 172.16.11.0/24: third octets 8, 9, 10, 11 are 00001000 to 00001011, so the first 6 bits of the third octet match: **172.16.8.0/22**.
- If 172.16.12.0/24 is added and summarised as **172.16.8.0/21**, the summary covers 172.16.8.0 to 172.16.15.255. That includes 172.16.13.0/24 to 172.16.15.0/24, which the router does not own. It attracts their traffic and drops it (black hole) or creates routing loops. Keep 172.16.8.0/22 and 172.16.12.0/24 separate.

Complete the code (`q2_vlsm.py`). Output matches the expected output exactly, ending with `next free address: 192.168.10.228`, and the `ipaddress` module confirms every network and broadcast address.

```
block = 1
while block < hosts + 2:
    block *= 2
prefix = 32 - int(math.log2(block))
first, last, bcast = addr + 1, addr + block - 2, addr + block - 1
print(f"{name} {to_ip(addr)}/{prefix} hosts {to_ip(first)} - {to_ip(last)} "
      f"bcast {to_ip(bcast)}")
addr += block
```

Alternative — smallest subnet first

If the allocator simply advances by the block size, the blocks overlap:

Step	Intended	Real network	Problem
WAN /30	192.168.10.0/30	192.168.10.0/30	fine
Library /27 at .4	192.168.10.4/27	192.168.10.0/27	**overlaps** the WAN link
Staff /26 at .36	192.168.10.36/26	192.168.10.0/26	**overlaps** both
Labs /25 at .100	192.168.10.100/25	192.168.10.0/25	**overlaps** all

A block must start on a multiple of its own size. If you align each block (WAN .0/30, Library .32/27, Staff .64/26, Labs .128/25) it works, but the addresses .4 to .31 (28 addresses) are wasted in fragments. Assigning the largest first keeps every block aligned, leaves no gaps, and leaves the free space in one block at the end.

Q3 — Which Way Will the Packet Go?

Longest-prefix match: the most specific matching route wins.

Destination	Matching routes	Winner	Next hop
10.1.2.200	/8, /16, /24, /25 (200 is in 128–255)	10.1.2.128/25	**192.0.2.13**
10.1.2.100	/8, /16, /24 (100 is not in 128–255)	10.1.2.0/24	**192.0.2.9**
10.1.9.9	/8, /16	10.1.0.0/16	**192.0.2.5**
10.7.7.7	/8	10.0.0.0/8	**192.0.2.1**
8.8.8.8	default only	0.0.0.0/0	**198.51.100.1**

R1 configuration with the TODOs completed

```
enable
configure terminal
hostname R1
interface GigabitEthernet0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
! TODO 1
exit
interface Serial0/0/0
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000
 no shutdown
exit
! TODO 2
ip route 192.168.20.0 255.255.255.0 10.0.0.2
! backup link (floating static route, administrative distance 200)
interface Serial0/0/1
 ip address 10.0.1.1 255.255.255.252
 clock rate 64000
 no shutdown
exit
ip route 192.168.20.0 255.255.255.0 10.0.1.2 200
end
write memory
show ip route
```

R2 configuration (reverse route)

```
enable
configure terminal
```

```
hostname R2
interface GigabitEthernet0/0
 ip address 192.168.20.1 255.255.255.0
 no shutdown
interface Serial0/0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
interface Serial0/0/1
 ip address 10.0.1.2 255.255.255.252
 no shutdown
exit
ip route 192.168.10.0 255.255.255.0 10.0.0.1
ip route 192.168.10.0 255.255.255.0 10.0.1.1 200
end
write memory
```

Expected result. `show ip route` on R1 shows `S 192.168.20.0/24 [1/0] via 10.0.0.2`; the backup route with distance 200 is not shown because it is not the best route. PC to PC pings work. **Test the backup:** `shutdown` the main serial interface; the table now shows `S 192.168.20.0/24 [200/0] via 10.0.1.2` and the ping continues (a few packets are lost during the switch). Remove the `clock rate` on the DCE end and the interface shows *up, line protocol down* and the link does not work. Save these `show` outputs and screenshots as your evidence.

Alternative — OSPF instead of static routes

Number of routers n	Static routes (one per remote LAN)	OSPF (3 lines per router)
2 campuses	2 × 1 = 2 lines (4 with floating backups)	6 lines
5 campuses	5 × 4 = 20 lines (40 with backups)	15 lines

Static lines grow with n × (n − 1); OSPF grows with n. Recovery: a floating static route reacts only when the local interface goes down (a few seconds) and not at all if the failure is further away; OSPF with default timers needs up to the 40 s dead interval, or 1 to 5 s on interface-down events and faster with tuned hello timers, and it also detects remote failures.

Q4 — Watch Distance-Vector Routing Count to Infinity

Round-by-round tables (from the program; columns A B C D E)

After round	A	B	C	D	E
1	0 2 5 8 99	2 0 3 4 12	5 3 0 1 5	8 4 1 0 4	99 12 5 4 0
2	0 2 5 6 12	2 0 3 4 8	5 3 0 1 5	6 4 1 0 4	12 8 5 4 0
3 (final)	0 2 5 6 10	2 0 3 4 8	5 3 0 1 5	6 4 1 0 4	10 8 5 4 0
4 (no change)	same	same	same	same	same

It converges after **4 rounds** (three rounds of change and one round confirming no change).

Why A's best route to C is not the direct link. Direct A–C costs 7; via B it costs A–B 2 + B–C 3 = **5**.

Count to infinity (X–Y–Z, cost 1 each, link Y–Z fails)

Round	Y's distance to Z	X's distance to Z
Before failure	1	2 (via Y)
1	3 (believes X's 2, +1)	2
2	3	4 (via Y's 3, +1)
3	5	4
4	5	6
…	keeps rising by 2 per two rounds	…
until	16 (RIP's "infinity")	16

X and Y keep passing the dead route to each other. **Split horizon** stops X from advertising Z back to Y (the route came from Y), so Y does not learn a false path. **Poison reverse** goes further: X advertises Z to Y with metric 16 (unreachable), so Y marks Z unreachable at once and tells X, and both converge in about 2 rounds instead of 15.

RIP in Packet Tracer (on each router)

```
router rip
 version 2
 no auto-summary
 network 192.168.10.0
 network 10.0.0.0
```

Use `show ip route` (R entries), then shut one link and record how long the table takes to change (RIP update timer 30 s, invalid 180 s, flush 240 s).

Complete the code (`q4_distance_vector.py`). Output: `converged after 4 rounds` and the matrix A 0 2 5 6 10 / B 2 0 3 4 8 / C 5 3 0 1 5 / D 6 4 1 0 4 / E 10 8 5 4 0, as expected.

```
new = link[u, n] + old[n][dest]
if new < table[u][dest]:
    table[u][dest] = new
    changed = True
```

C–D failed (cost 99), program rerun (`python3 q4_distance_vector.py fail`): converged after **3 rounds**.

	A	B	C	D	E
A	0	2	5	10	14
B	2	0	3	8	12
C	5	3	0	11	15
D	10	8	11	0	4
E	14	12	15	4	0

Alternative — link-state routing after C–D fails

In my own simulation, C and D each flood one link-state advertisement over the five remaining links. Each advertisement needs about **6 transmissions**, so about **12 messages** in total, and reaches every router within **3 hops** (C → B → D → E). Each router then runs Dijkstra locally, so every router has a correct route after about 3 flooding steps plus the SPF calculation. Distance vector with triggered updates took **about 4 rounds and about 15 vector messages** in my simulation (more if full tables are sent periodically), and it can count to infinity in other topologies. Link-state is faster and safer, but needs more memory and CPU.

Q5 — Why Does OSPF Treat 1 Gbps and 10 Gbps the Same?

Cost = reference bandwidth / link bandwidth (minimum 1)

Link speed	Default reference 100 Mbps	New reference 100 Gbps
10 Mbps	10	10 000
100 Mbps	1	1 000
1 Gbps	0.1, rounded to **1**	100
10 Gbps	0.01, rounded to **1**	10

The problem. With the default reference, 100 Mbps, 1 Gbps and 10 Gbps links all cost 1, so a path over a 1 Gbps link ties with, or beats, a path over 10 Gbps. **Fix:** set a reference bandwidth of 100 Gbps (`auto-cost reference-bandwidth 100000` on every router) so that 10 Gbps costs 10, 1 Gbps costs 100, and speeds up to 100 Gbps are distinguished.

My 6-router network. Links (cost with the new reference): R1–R2 10 G (10), R2–R4 10 G (10), R4–R6 10 G (10), R1–R3 1 G (100), R3–R5 1 G (100), R5–R6 1 G (100), R2–R3 1 G (100), R4–R5 1 G (100).

Dijkstra from R1

Step	Settled router	Tentative distances afterwards
1	R1 (0)	R2 = 10, R3 = 100
2	R2 (10)	R4 = 20; R3 stays 100 (10 + 100 = 110 is worse)
3	R4 (20)	R6 = 30, R5 = 120
4	R6 (30)	nothing improves
5	R3 (100)	R5 stays 120 (100 + 100 = 200 is worse)
6	R5 (120)	done

Shortest-path tree: R1 → R2 (10) → R4 (20) → R6 (30), R4 → R5 (120), and R1 → R3 (100). With the default reference all costs are 1, R1 reaches R6 in 3 hops over either path, so the 1 Gbps path ties with the 10 Gbps path. With the new reference the **10 Gbps path (cost 30) is chosen over the 1 Gbps path (cost 300)**.

Packet Tracer (on each router)

```
router ospf 1
 auto-cost reference-bandwidth 100000
```

```
network 10.0.12.0 0.0.0.3 area 0
network 10.0.24.0 0.0.0.3 area 0
```

If Packet Tracer does not take the interface speed into account, set the cost on each interface with `ip ospf cost 10` (10 G) or `ip ospf cost 100` (1 G). Check with `show ip ospf interface` and `show ip route`, and record the chosen path before and after.

One area or several. For this six-router lab one area is enough. For a state internet provider, use **several areas** (area 0 as backbone, one area per city or region) because it limits the size of the link-state database, keeps flooding and SPF runs local, and allows route summarisation at the area border routers. Rule of thumb: stay with a single area up to a few tens of routers.

Alternative — EIGRP or IS-IS

- EIGRP (classic) metric = 256 × (10^7 / lowest bandwidth in kbps + total delay in tens of µs). With 10 µs delay: 1 Gbps = 256 × (10 + 1) = **2 816**; 10 Gbps = 256 × (1 + 1) = **512**. EIGRP already prefers the 10 Gbps path with no change of reference bandwidth. (The classic formula cannot separate speeds above 10 Gbps; wide metrics fix that.)
- IS-IS gives every interface a default metric of **10**, so 1 G and 10 G tie, and metrics must be set manually, like OSPF.
- Decision: EIGRP avoids this problem without changes, but it is Cisco-oriented; for a multi-vendor ISP, OSPF with a corrected reference bandwidth is the better choice.

Q6 — IPv6, ICMP, Multicast and SDN in a Smart City

IPv6 shortest form. `2001:0db8:0000:0000:0000:ff00:0042:8329` becomes **`2001:db8::ff00:42:8329`** (drop leading zeros; replace the longest run of zero groups with `::` once).

EUI-64 for MAC 00:1A:2B:3C:4D:5E

1. Split the MAC: 00:1A:2B | 3C:4D:5E.
2. Insert FF:FE in the middle: 00:1A:2B:**FF:FE**:3C:4D:5E.
3. Flip the universal/local bit (bit 7 of the first byte): 00 → **02**.
4. Interface identifier = **021A:2BFF:FE3C:4D5E** (link-local address `fe80::21a:2bff:fe3c:4d5e`).

Traceroute with ICMP Time Exceeded. traceroute sends probes with TTL = 1, 2, 3, … Each router reduces the TTL by 1. The router that makes it 0 discards the packet and returns an ICMP *Time Exceeded* message (type 11) from its own address, so the sender learns each router in turn. When the probe finally reaches the destination, the destination answers (ICMP port unreachable or echo reply), and traceroute stops.

Multicast MAC for 239.1.2.3. Take 01:00:5e and the lower 23 bits of the group address (1.2.3 = 01:02:03): **01:00:5e:01:02:03**. The camera sends **one** stream to the group

address. Receivers (control rooms) join with IGMP, and routers (PIM) and switches (IGMP snooping) copy the stream only where the tree branches, so the camera never makes several copies.

OpenFlow rules (highest priority first)

- HTTP packet to 10.5.1.1: it matches priority 200 (destination 10.5.0.0/16) and priority 100 (TCP port 80). The highest priority wins, so it is **forwarded out of port 3**, not sent to the controller.
- SSH (TCP 22) packet to 10.6.1.1: it matches neither rule 200 (10.6 is not 10.5) nor rule 100 (port 22 is not 80), so it falls to priority 0 and is **dropped**.

Is SDN worth adopting? For a city with 2 million sensors and central routing wishes: yes, in stages. Benefits are central policy changes, a global view and faster changes. Risks are the controller as a single point of failure (use redundant controllers), controller scalability and security, and staff skills. Start with the camera and core network, and keep conventional routing as a fallback.

Alternative — native IPv6, dual-stack, or 6in4 tunnel

- A 6in4 tunnel adds a 20-byte IPv4 header: 100 → **120 bytes (+20 %)** for a 100-byte sensor packet, plus tunnel-endpoint processing.
- Dual-stack has no tunnel overhead but means running and securing two protocols.
- Going straight to IPv6 solves the address shortage for 2 million sensors, but needs every device and service to support it (or NAT64/DNS64).
- Decision: **start dual-stack in the core, make sensor networks IPv6-only**, and use tunnels only as a short-term bridge across IPv4-only links.

Q7 — Find the Weak Point, Redesign and Validate

Chosen network: the college campus of Q2 (192.168.10.0/24, VLSM, growing). Replace the numbers with your own if you choose a different network. Run the original and the redesign in Packet Tracer and save both results.

A. Identify the limitation

- The VLSM plan fits 100 + 50 + 25 + 2 hosts exactly, using 228 of 256 addresses with 28 left over in fragments (.228/30, .232/29, .240/28).
- Adding the 60-host hostel needs 64 addresses, and total demand is now **292 > 256**. The largest free block is a /28 (16 addresses). The cause is the addressing design: one fixed /24 with no growth room and no planned summarisation.

B. Measurable objective

1. Fit the hostel (60 hosts) and 5 years of growth (100 % in each subnet) with **0 renumbered subnets**.
2. Keep at least **25 % of the address space free**.

3. Advertise the campus as **one summary route** (1 route instead of 5).

C. Revised network

Use **192.168.10.0/23** (512 addresses) with VLSM, largest first, and keep the unused blocks free for growth:

Subnet	Hosts now	Block	Network	Usable hosts
Labs	100	/25	192.168.10.0/25	126
Hostel (new)	60	/26	192.168.10.128/26	62
Reserved for growth	—	/26	192.168.10.192/26	62
Staff	50	/26	192.168.11.0/26	62
Library	25	/27	192.168.11.64/27	30
WAN link	2	/30	192.168.11.96/30	2
Free	—	—	192.168.11.100 to 192.168.11.255	—

Allocated: 128 + 64 + 64 + 32 + 4 = 292 of 512 addresses, so **220 addresses (43 %) stay free**. Trade-offs: a larger address block (192.168.10.0/23) and a larger summary to document; benefit: the hostel fits with no renumbering, and the whole campus is advertised as one summary route, 192.168.10.0/23. Draw the topology (router, switches, one VLAN per subnet) and attach this address table.

D. Predict before implementing

Prediction: the hostel connects with **0 renumbered subnets**, free addresses rise from 28 (11 % of 256) to **220 (43 % of 512)**, and the routing table needs **1 summary route** instead of 5. Pings between all subnets succeed in both versions.

E. Implement and validate (your Packet Tracer work)

Run the same tests on both versions: (1) `ping` from every subnet to every other, (2) `show ip route` and count the routes, (3) count addresses allocated and free, (4) try to add the hostel. Expected: original — hostel does not fit, 5 routes; redesign — hostel fits, 1 summary route. Save `.pkt` files, configs and screenshots for both.

F. Defend the final design

Judge whether each measurable objective was met (renumbered subnets = 0, free space above 25 %, one summary route). Costs: a larger address plan and more care when documenting it. The original may still be preferable for a small fixed network that will never grow, because it is simpler and wastes no addresses.